C08 # Surface d'Attaque & Menaces

Conception Intermediate

Comprendre les vecteurs d'attaque spécifiques à Claude Code

Les 4 vecteurs principaux

Claude Code est un agent avec accès au filesystem, aux commandes Bash et à Internet. Sa surface d'attaque est plus large qu'un simple assistant textuel.

Vecteur 1 : Prompt injection via contenu externe. Quand Claude lit un email, une issue GitHub ou un fichier analysé contenant des instructions malveillantes, il peut les exécuter. L'attaque exploite la confusion entre données et instructions.

Vecteur 2 : Exfiltration de secrets via Bash. Si les permissions sont trop larges, un prompt malveillant peut demander à Claude de lire `.env`, les clés SSH ou les tokens stockés localement, puis d'exfiltrer via une requête HTTP.

Vecteur 3 : Supply chain via skills et plugins malveillants. Une étude Snyk (ToxicSkills, 2026) sur 3 984 skills a trouvé 36,8% de skills avec des failles de sécurité, dont 534 à risque critique. Le « rug pull » est la variante la plus insidieuse : un MCP légitime qui devient malveillant après avoir été approuvé.

Vecteur 4 : Escalade de privilèges dans les pipelines multi-agents. Dans une orchestration, un agent compromis peut passer des instructions malveillantes aux agents suivants. Les outputs d'un agent deviennent les inputs de l'autre sans validation intermédiaire.

Matrice de risque rapide

Scénario	Risque	Action immédiate
Solo dev, repos publics	Moyen	Installer un hook output-scanner
Équipe, codebase sensible	Élevé	Vetting MCPs + hooks injection
Enterprise, production	Critique	Moindre privilège, isolation et vérification d'intégrité

CVEs à connaître (sélection 2025-2026)

CVE	Sévérité	Résumé
CVE-2025-53109/53110	High	Sandbox escape filesystem MCP
CVE-2025-54135	High	RCE via prompt injection dans Cursor
ADVISORY-CC-2026-001	High	Sandbox bypass, patcher v2.1.34+
ADVISORY-CC-2026-002	Medium	Deny rules ignorées au-delà de 50 sous-commandes, patcher v2.1.90+
CVE-2026-0755	Critical (9.8)	RCE dans gemini-mcp-tool (pas de patch)

Action immédiate si vous êtes en v2.1.33 ou antérieur : mettez à jour vers v2.1.34+ pour corriger le sandbox bypass.

Défense en profondeur

Le principe est simple : chaque couche réduit la probabilité qu'une attaque aboutisse.

```
Permissions minimales (settings.json)
  + Whitelist d'outils par tâche
    + Lecture des MCPs avant installation
      + Logs JSONL pour détection post-incident
        + Sandbox / environnement éphémère
```

Règle de base sur les permissions : accorder uniquement ce que la tâche nécessite. Pour une analyse, limiter l'agent avec `tools: Read, Grep, Glob` ou `--tools "Read,Grep,Glob"`. Une règle allow pré-approuve un outil ; elle ne retire pas les autres.

Audit avec les logs JSONL

Claude Code écrit tous ses tool calls dans le fichier JSONL de la session, `~/.claude/projects/{chemin-encodé}/{session-id}.jsonl`. Ces logs permettent de détecter des comportements anormaux après coup : lectures inattendues de fichiers sensibles, appels réseau non prévus, tentatives de modification hors scope.

```
# Inspecter les tool calls d'une session
cat ~/.claude/projects/*/*.jsonl | jq '.message.content[]?.name // empty'
```